Lecture 6: Arrays and Advanced C Programming

Objective:
Explore advanced C programming concepts like arrays, structures, pointers, and memory management, all of which are foundational in High-Performance Computing (HPC).

Duration: 1 Hour

🎯 Learning Outcomes

By the end of this lecture, students will be able to:

- Declare and use single- and multi-dimensional arrays in C
- Understand the use and structure of `struct` in C programming
- Use pointers to manipulate data and memory addresses
- Dynamically allocate and free memory using `malloc()` and `free()`

🧩 1. Arrays in C

📌 Declaration & Initialization

```
int numbers[5];                    // Declaration
int marks[3] = {90, 85, 75};       // Declaration + Initialization
```

🔁 Accessing Array Elements

```
for (int i = 0; i < 3; i++) {
    printf("%d\n", marks[i]);
}
```

📌 Multi-dimensional Arrays

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

```
printf("%d", matrix[1][2]);  // Output: 6
```

📝 **Use in HPC:** Arrays store large data like simulation results, matrices for scientific computing, etc.

🏗️ 2. Structures in C

📌 Definition and Usage

```
struct Student {
    int id;
    float marks;
    char name[30];
};

struct Student s1 = {101, 89.5, "Ankit"};

printf("Name: %s, Marks: %.2f", s1.name, s1.marks);
```

🧠 Why Structures?

- Combine variables of different types into a single logical unit.
- Common in HPC for storing metadata, records, or node parameters.

🧭 3. Pointers in C

📌 Basic Pointer Concept

```
int a = 10;
int *p;
p = &a;

printf("Address: %p, Value: %d", p, *p);
```

- `*p` – Dereferencing (access the value at address)
- `&a` – Address-of operator

🔁 Pointer with Arrays

```
int arr[3] = {1, 2, 3};
int *ptr = arr;

for (int i = 0; i < 3; i++) {
    printf("%d ", *(ptr + i));
}
```

📌 **Use in HPC:** Pointers enable direct memory access, leading to faster data manipulation.

🧠 4. Dynamic Memory Allocation

📌 Using `malloc()` and `free()`

```
int *ptr;
ptr = (int*) malloc(5 * sizeof(int));  // Allocate memory for 5 integers

if (ptr == NULL) {
    printf("Memory allocation failed");
} else {
    for (int i = 0; i < 5; i++) {
        ptr[i] = i + 1;
        printf("%d ", ptr[i]);
    }
    free(ptr);  // Free allocated memory
}
```

📌 **Use in HPC:** Required when memory demands change during runtime (e.g., adaptive mesh refinement).

🔧 Classroom Activities

🔸 Activity 1: Array Practice

- Write a C program to input 5 integers into an array and print the sum.

🔸 Activity 2: Structure Assignment

- Define a `struct` for a Book with title, author, and price.
- Create 2 books and display them.

🔸 Activity 3: Pointer Quiz

- Given a code snippet, predict the output:

```
c
CopyEdit
int x = 20; int *p = &x;
*p = 15; printf("%d", x);
```

🔸 Lab Integration (if applicable)

- Code a matrix addition using 2D arrays.
- Allocate memory dynamically for an array of floats and compute their average.

📚 Summary

Concept	Description
Arrays	Store data in indexed form; used for batch data
Structures	Combine multiple data types into a single entity
Pointers	Access and modify memory directly
malloc/free	Allocate/release memory during runtime

🔁 **In HPC**, memory efficiency and direct data control via pointers/structures are **critical** to performance and scalability.

📝 Homework/Assignment

Task:
Write a C program that:

1. Uses `malloc()` to allocate memory for 10 integers
2. Accepts user input for those 10 values
3. Calculates and prints their sum
4. Frees the memory at the end